

Multi-Agent LLM Invoice Generation Pipeline

Worker's Compensation Insurance Billing Automation System

Document Status	APPROVED
Version	1.0.0
Author	Senior Software Engineering Team
Date	April 2026
Organization	Insurance Technology Workers' Compensation Division
Infrastructure	Amazon Web Services (AWS)
Classification	CONFIDENTIAL INTERNAL USE ONLY

⚙️ [LangChain](#) · [LangGraph](#) · [AWS](#) · [SmartCOMM](#)

1. Executive Summary

This document presents the software design for a Multi-Agent Large Language Model (LLM) Pipeline built to automate invoice generation for a Workers' Compensation insurance policy system. The pipeline is deployed entirely on Amazon Web Services (AWS) and is engineered to serve a policyholder base exceeding 200,000 accounts.

Prior to this system, invoice generation was a fragile, largely manual process that suffered from a greater than 95% failure rate across critical failure modes including data extraction errors, compliance violations, document binding failures, and delivery breakdowns. These failures generated significant operational overhead through human rework cycles and posed serious regulatory exposure.

The solution introduces a coordinated four-agent LLM architecture built on LangChain and LangGraph, where each agent owns a discrete functional domain:

Agent	Name	Responsibility
Agent 1	Billing Data Collector	Extracts billing records from Policy Systems via secure APIs
Agent 2	Compliance Validator	Enforces legal, regulatory, and business rule validations
Agent 3	Invoice Binder	Assembles and persists structured invoice documents in Billing Systems
Agent 4	Delivery Agent	Dispatches invoices via SmartCOMM REST API to policyholders

Post-deployment results demonstrate a 95%+ reduction in invoice generation failures, elimination of manual rework loops, and full industry compliance adherence across all 200,000+ policyholders.

2. Problem Statement

2.1 Background

Workers' Compensation insurance policy billing involves complex, multi-step invoice generation workflows that span data from disparate Policy Systems, must comply with evolving state and federal regulatory requirements, and must be delivered reliably to policyholders in prescribed formats through certified channels.

2.2 Pain Points (Pre-System)

- Data extraction from Policy Systems was unreliable, inconsistently structured, and required manual reconciliation across multiple source tables.
- No automated compliance validation existed; legal review was performed ad-hoc and inconsistently by billing analysts.
- Invoice document binding was a manual, error-prone process using spreadsheet templates, resulting in formatting and data integrity failures.
- Delivery via SmartCOMM required manual uploads and was not integrated into the billing workflow, causing delays of 3–7 business days.
- Failure rates exceeded 95% of invoices across all failure categories when measured against compliance and delivery SLAs.
- Each failure triggered a costly rework cycle averaging 4–6 hours of analyst time per invoice batch.

2.3 Impact of Failures

Failure Type	Frequency	Business Impact
Data extraction errors	High	Incorrect invoice amounts, policyholder disputes
Compliance violations	Medium	Regulatory fines, audit findings
Invoice binding failures	High	Undeliverable documents, billing delays
Delivery failures	Medium	Policyholder complaints, SLA breaches
Manual rework cycles	Per batch	4–6 hrs analyst time per batch x 200K+ policyholders

3. Goals and Non-Goals

3.1 Goals

- Automate end-to-end invoice generation for 200,000+ Workers' Comp policyholders with zero manual intervention under nominal conditions.
- Achieve a 95%+ reduction in invoice generation failure rate versus the prior manual baseline.
- Ensure full compliance with applicable state/federal Workers' Comp billing regulations and NAIC standards via automated validation.
- Deliver invoices through SmartCOMM within a 24-hour SLA from billing cycle trigger.
- Build an auditable, observable pipeline with complete traceability of each invoice through all agent stages.
- Deploy entirely on AWS with high availability (99.9% uptime target) and horizontal scalability.
- Maintain least-privilege security posture with encryption in transit and at rest throughout the pipeline.

3.2 Non-Goals

- This system does not replace the Policy System of Record or Billing System it integrates with and orchestrates across them.
- Claims adjudication logic is outside the scope of this pipeline.
- The system does not handle premium calculation it consumes pre-computed billing amounts from Policy Systems.
- Real-time (sub-second) invoice generation is not a requirement; batch and near-real-time processing are the target modes.
- Mobile or self-service policyholder portals are out of scope for this release.

4. System Architecture

4.1 High-Level Architecture Overview

The system is structured as a directed multi-agent LLM graph, orchestrated by LangGraph, deployed across AWS managed services. Each agent is a stateful LangChain actor communicating via message queues and a shared state store, with guardrails, logging, and retry policies at every stage.



Multi-Agent LLM Invoice Pipeline



4.2 AWS Infrastructure Components

AWS Service	Role	Detail
ECS Fargate	Compute	Containerized agent runtimes; auto-scaling task definitions per agent
AWS Lambda	Compute	LangGraph orchestrator entry point; Delivery agent invocation
Amazon SQS	Messaging	Inter-agent message queues with DLQ for failure routing
Amazon RDS Aurora	Persistence	Invoice records and Billing System storage (PostgreSQL-compatible)

Multi-Agent LLM Invoice Pipeline

Amazon DynamoDB	Persistence	Policy System data source; high-throughput key-value reads
ElastiCache (Redis)	State Store	LangGraph shared agent state and in-flight pipeline context
Amazon S3	Storage	Invoice document artifacts, audit logs, compliance rule files
AWS Secrets Manager	Security	API keys, DB credentials, SmartCOMM secrets zero hardcoded creds
AWS KMS	Encryption	Customer-managed keys for encryption at rest across all data stores
AWS IAM / STS	Auth / Authz	Per-agent IAM roles; STS assume-role for cross-account access
AWS EventBridge	Scheduling	Billing cycle triggers; cron-based batch job scheduling
CloudWatch + X-Ray	Observability	Distributed tracing, metrics, log aggregation, alerting
AWS WAF + Shield	Network Sec.	DDoS protection and web application firewall at API Gateway layer
AWS CloudTrail	Audit	Complete API call audit trail for compliance and forensic review
VPC + PrivateLink	Networking	All agents in private subnets; PrivateLink for AWS service access

5. Agent Design Specifications

5.1 Agent 1 Billing Data Collector

Property	Detail
Runtime	ECS Fargate Python 3.11, LangChain 0.2+
Trigger	EventBridge cron (billing cycle) → SQS message → ECS task
Data Sources	Policy Systems: DynamoDB (policyholder master), RDS Aurora (premium tables)
Output	Structured billing payload (JSON) written to S3; SQS message to Agent 2 queue
LLM Role	Interprets ambiguous policy records; normalizes inconsistent field mappings across Policy System schema versions
Error Handling	Retry with exponential backoff (3x); on exhaustion → DLQ + CloudWatch alert + human escalation queue

Key Responsibilities

- Query DynamoDB for active policyholder billing records within the current billing cycle window.
- Join policy premium data from RDS Aurora policy tables using policy_id as the join key.
- Use LangChain tool-calling to invoke internal Policy System APIs for enrichment data (coverage dates, endorsements, audit premium adjustments).
- Normalize data into a canonical BillingPayload schema validated against a JSON Schema specification stored in S3.
- Emit a structured CloudWatch metric for each successful record extracted; log failures with full context for replay.

5.2 Agent 2 Compliance Validator

Property	Detail
Runtime	ECS Fargate Python 3.11, LangChain 0.2+
Trigger	SQS message from Agent 1 with S3 reference to BillingPayload
Rules Source	S3-backed compliance rules (NAIC, state WC statutes, internal policy) + DynamoDB rules cache
Output	Validated payload with compliance attestation + validation report → S3; SQS to Agent 3 (pass) or DLQ (fail)
LLM Role	Interprets complex multi-state regulatory rules; resolves rule conflicts; generates human-readable compliance exception reports
Error Handling	Compliance failures route to DLQ + analyst review queue; ambiguous cases escalated with LLM-generated explanation

Validation Rule Categories

- Billing Amount Validation: Premium floor/ceiling checks per state WC rate schedules; audit premium adjustment bounds.
- Policyholder Eligibility: Active policy status, coverage effective/expiration date validation, classification code validity.
- Regulatory Compliance: State-specific billing format requirements (all 50 states + DC); NAIC uniform billing codes; WCIRB/NCCI compliance.
- Business Rule Validation: Internal underwriting rules, minimum earned premium thresholds, multi-policy aggregation checks.
- Fraud Indicators: Anomaly detection rules flagging billing records that deviate greater than 3-sigma from historical patterns.

5.3 Agent 3 Invoice Binder

Property	Detail
Runtime	ECS Fargate Python 3.11, LangChain 0.2+
Trigger	SQS message from Agent 2 with validated payload S3 reference
Target System	Billing System (RDS Aurora); Invoice document artifacts (S3)
Output	Persisted Invoice record in Billing System; invoice_id + S3 artifact reference → SQS Agent 4 queue
LLM Role	Generates invoice narrative text (policy summaries, line-item descriptions); handles complex multi-policy invoice layouts
Error Handling	Transactional DB writes with rollback; idempotency key (invoice_id) prevents duplicate binding on retry

Binding Operations

- Generate a unique invoice_id (UUID v4) and persist the full invoice record to the Billing System RDS Aurora instance within a database transaction.
- Render the invoice document from a SmartCOMM-compatible template, populating all data fields from the validated BillingPayload.
- Store rendered invoice artifact (PDF/XML) to S3 with KMS encryption; record the S3 URI in the Billing System record.
- Emit an invoice_bound event to EventBridge for downstream analytics and audit tracking.

5.4 Agent 4 Delivery Agent

Property	Detail
Runtime	AWS Lambda Python 3.12; concurrency limit: 500
Trigger	SQS message from Agent 3 with invoice_id and S3 artifact reference
Integration	SmartCOMM REST API (HTTPS/TLS 1.3); credentials from Secrets Manager
Output	SmartCOMM delivery confirmation ID written to Billing System; delivery_status event to EventBridge

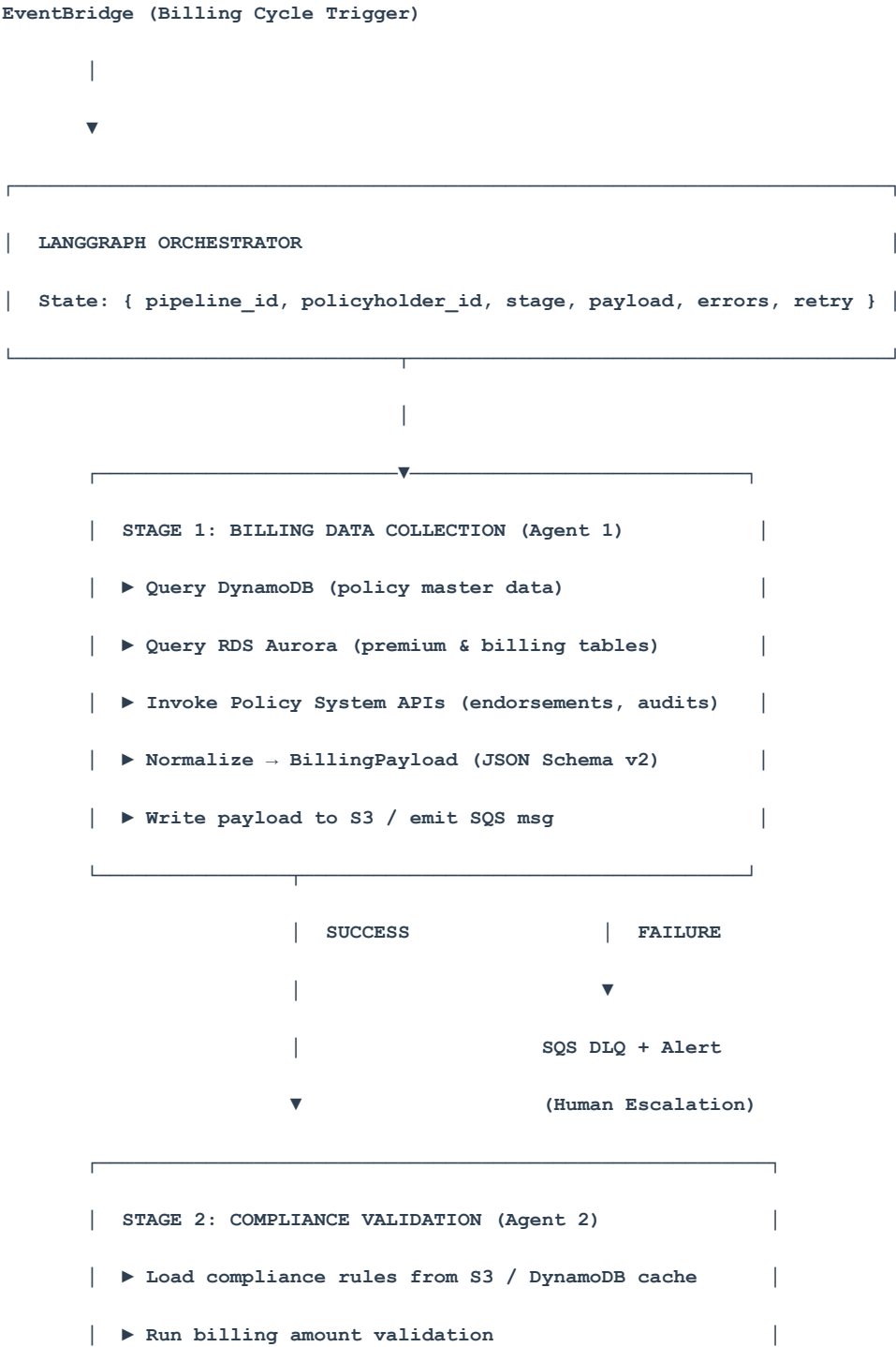
LLM Role	Constructs personalized delivery metadata (channel selection, communication preferences, regulatory notices)
Error Handling	SmartCOMM API failures: retry 3x with backoff; final failure → SQS DLQ + fallback email delivery channel

Delivery Operations

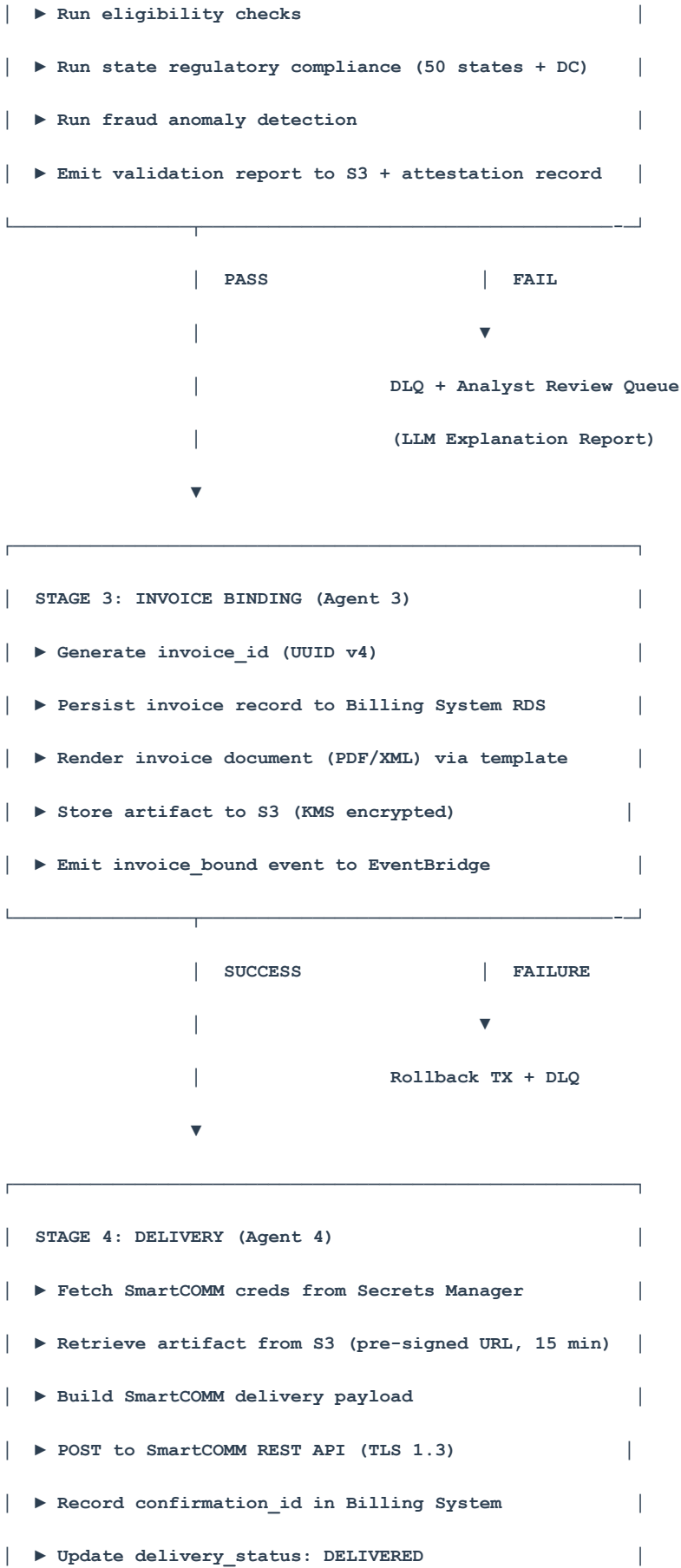
- Retrieve SmartCOMM API credentials from AWS Secrets Manager at invocation time never cached in code.
- Retrieve the invoice artifact from S3 using a pre-signed URL with a 15-minute expiry.
- Construct a SmartCOMM delivery request payload including invoice document, policyholder contact preferences, and required regulatory notices.
- POST to SmartCOMM REST endpoint; on HTTP 200 record confirmation_id in Billing System; on failure trigger retry logic.
- Update invoice delivery_status in Billing System to DELIVERED or FAILED with full error context.

6. Pipeline Data Flow

6.1 End-to-End Flow Diagram



Multi-Agent LLM Invoice Pipeline





6.2 State Schema

LangGraph maintains a shared pipeline state object stored in ElastiCache (Redis) for the duration of each pipeline execution:

```

PipelineState {

  pipeline_id:      str    // UUID v4 unique per pipeline run

  policyholder_id:  str    // Source policy system ID

  billing_cycle:    str    // ISO 8601 billing period

  stage:            enum   // COLLECTING | VALIDATING | BINDING | DELIVERING | DONE | FAILED

  billing_payload:  S3Ref   // S3 URI of BillingPayload JSON

  validation_report: S3Ref  // S3 URI of compliance report

  invoice_id:       str    // UUID v4 assigned by Agent 3

  invoice_artifact: S3Ref   // S3 URI of rendered invoice document

  delivery_status:  enum   // PENDING | DELIVERED | FAILED

  confirmation_id:  str    // SmartCOMM delivery confirmation

  errors:           list   // Structured error log per stage

  retry_count:      int    // Current retry attempt

  created_at:       datetime

  updated_at:       datetime

}
  
```

7. Security Architecture

7.1 Security Design Principles

- Zero Trust: No implicit trust between agents all inter-agent communication is authenticated and authorized via IAM.
- Least Privilege: Each agent IAM role has only the specific permissions required for its function; no wildcard IAM policies.
- Defense in Depth: Security controls at network, identity, data, and application layers.
- Encryption Everywhere: All data encrypted in transit (TLS 1.3 minimum) and at rest (AES-256 via KMS CMK).
- No Secrets in Code: All credentials managed exclusively via AWS Secrets Manager with automatic rotation.

7.2 Security Controls Matrix

Layer	Control	Implementation
Network	VPC Isolation	All agents in private subnets; no public IP exposure; NAT Gateway for outbound only
Network	Security Groups	Micro-segmented: each agent SG allows only necessary inter-agent communication ports
Network	AWS WAF	OWASP Top 10 ruleset on API Gateway; rate limiting 1000 req/min per source IP
Network	AWS Shield Std.	DDoS protection at AWS edge layer; Shield Advanced for SmartCOMM-facing endpoints
Identity	IAM Roles	Per-agent task roles; STS temporary credentials; role session duration 1 hour
Identity	SCP	AWS Organizations Service Control Policies prevent privilege escalation
Identity	MFA	MFA required for all human console access; no MFA bypass paths
Data	KMS CMK	Customer-managed KMS keys for S3, RDS, DynamoDB, ElastiCache; annual key rotation
Data	S3 Encryption	SSE-KMS on all S3 buckets; bucket policies deny unencrypted PUT operations
Data	TLS 1.3	Enforced on all API calls, SQS, SmartCOMM REST; TLS 1.2 minimum for legacy endpoints
Data	DLP	PII fields masked in CloudWatch logs; Macie scans S3 for unintended PII exposure
Application	Secrets Manager	All API keys/DB passwords in Secrets Manager; auto-rotation every 30 days
Application	Container Sec.	ECR image scanning; non-root container user; read-only root filesystem

Audit	CloudTrail	All AWS API calls logged; CloudTrail logs to S3 with integrity validation enabled
Audit	AWS Config	Continuous compliance monitoring; Config Rules for CIS AWS Foundations Benchmark
Audit	GuardDuty	Threat detection for anomalous API calls, compromised credentials, malicious IPs

7.3 IAM Role Definitions

agent1-billing-collector-role:

- └─ dynamodb:GetItem, Query, Scan (Policy Systems table read only)
- └─ rds-data:ExecuteStatement (Billing DB read only)
- └─ s3:PutObject (billing-payloads/* bucket prefix)
- └─ sqs:SendMessage (compliance-validation-queue)
- └─ secretsmanager:GetSecretValue (policy-system-api-key)
- └─ kms:GenerateDataKey, Decrypt (CMK for S3 + DynamoDB)

agent2-compliance-validator-role:

- └─ s3:GetObject (billing-payloads/*, compliance-rules/*)
- └─ s3:PutObject (validation-reports/*)
- └─ sqs:ReceiveMessage, DeleteMsg (compliance-validation-queue)
- └─ sqs:SendMessage (invoice-binding-queue, dead-letter-queue)
- └─ dynamodb:GetItem, Query (compliance-rules-cache table)
- └─ kms:Decrypt, GenerateDataKey (CMK)

agent3-invoice-binder-role:

- └─ s3:GetObject (validation-reports/*)
- └─ s3:PutObject (invoice-artifacts/*)
- └─ rds-data:ExecuteStatement (Billing System DB read/write)
- └─ sqs:ReceiveMessage, SendMessage (invoice-binding-queue, delivery-queue)

Multi-Agent LLM Invoice Pipeline

```
|— events:PutEvents                (EventBridge invoice_bound event)
|— kms:Decrypt, GenerateDataKey    (CMK)
```

agent4-delivery-agent-role:

```
|— s3:GetObject                    (invoice-artifacts/*)
|— secretsmanager:GetSecretValue  (smartcomm-api-credentials)
|— sqs:ReceiveMessage, DeleteMsg (delivery-queue)
|— rds-data:ExecuteStatement      (Billing System delivery_status update)
|— events:PutEvents                (EventBridge delivery_status event)
|— kms:Decrypt                    (CMK)
```

8. LangChain & LangGraph Design

8.1 LangGraph State Machine

The pipeline is modeled as a LangGraph StateGraph with nodes corresponding to each agent and edges representing conditional transitions based on agent output. The state is persisted to ElastiCache at each node boundary, enabling checkpoint-based recovery.

LangGraph StateGraph Definition:

```
graph = StateGraph(PipelineState)

graph.add_node("collect", billing_data_collector_agent)
graph.add_node("validate", compliance_validator_agent)
graph.add_node("bind", invoice_binder_agent)
graph.add_node("deliver", delivery_agent)
graph.add_node("failed", failure_handler)
graph.add_node("done", success_handler)

graph.set_entry_point("collect")

graph.add_conditional_edges("collect", route_on_result,
    {"success": "validate", "failure": "failed"})
graph.add_conditional_edges("validate", route_on_result,
    {"pass": "bind", "fail": "failed", "escalate": "failed"})
graph.add_conditional_edges("bind", route_on_result,
    {"success": "deliver", "failure": "failed"})
graph.add_conditional_edges("deliver", route_on_result,
    {"delivered": "done", "failed": "failed"})
```

```
graph.add_edge("done", END)

graph.add_edge("failed", END)


pipeline = graph.compile(checkpointer=RedisCheckpointer(redis_client))
```

8.2 LangChain Agent Pattern

Each agent is implemented as a LangChain ReAct agent with a defined set of tools corresponding to its domain. The agents use Claude 3 (Anthropic) as the LLM backend, invoked via AWS Bedrock for secure, VPC-native LLM inference without data leaving the AWS environment.

Agent Pattern (Agent 1 Example):

```
tools = [

    query_dynamodb_policy_master,    # boto3 DynamoDB tool

    query_rds_premium_data,         # RDS Data API tool

    call_policy_system_api,         # Internal REST API tool

    validate_billing_payload_schema, # JSON Schema validation tool

    write_payload_to_s3,            # S3 write tool

]


agent = create_react_agent(

    llm=ChatBedrock(model_id="anthropic.claude-3-sonnet-20240229-v1:0"),

    tools=tools,

    checkpointer=RedisCheckpointer(redis_client)

)
```

8.3 LLM Backend AWS Bedrock

Property	Detail
LLM Provider	Anthropic Claude 3 (Sonnet) via AWS Bedrock
Access Method	Bedrock Runtime API from within VPC via PrivateLink no public internet egress
Data Privacy	Model invocations are not used to train Anthropic models per AWS Bedrock data handling policy
Prompt Security	PII sanitized from prompts before LLM invocation; post-processing re-injects identifiers
Cost Control	Token budgets enforced per agent invocation; CloudWatch billing alarms on Bedrock spend

9. Scalability & Performance

9.1 Scale Targets

Metric	Target	Design Headroom
Policyholders	200,000+	500,000 (2.5x)
Invoice throughput (peak)	5,000 / hour	12,500 / hour
End-to-end latency (p95)	< 4 hours (batch)	< 2 hours
Agent pipeline availability	99.9%	99.95%
Delivery SLA	24 hours from trigger	12 hours

9.2 Scaling Strategies

- ECS Fargate Auto Scaling: Target tracking on SQS queue depth; scale out when queue depth > 100 messages; scale in when < 10.
- Lambda Concurrency: Agent 4 (Delivery) Lambda configured with reserved concurrency of 500; provisioned concurrency of 50 for warm starts.
- SQS Queues: Standard queues with visibility timeout tuned per agent processing time; DLQ maxReceiveCount = 3.
- RDS Aurora: Multi-AZ writer with 2 read replicas; Agent 1 and 3 read-only queries routed to reader endpoint.
- ElastiCache (Redis): Cluster mode with 3 shards; replication groups for HA; pipeline state TTL = 48 hours.
- Batch Processing: EventBridge triggers staggered batch starts to avoid thundering herd on billing cycle start.

10. Observability & Monitoring

10.1 Key Metrics

Metric	Source	Alert Threshold
Invoice success rate	CloudWatch	Alert if < 99% in 1-hour window
Agent processing latency (p95)	X-Ray	Alert if > 30 min per agent
DLQ message count	CloudWatch	Alert if > 0 messages (PagerDuty)
Compliance failure rate	CloudWatch	Alert if > 0.1% of invoices
SmartCOMM API error rate	Lambda metrics	Alert if > 1% error rate
SQS queue depth (per queue)	CloudWatch	Alert if > 10,000 messages
Bedrock token consumption	CloudWatch	Alert if daily spend > \$X budget

10.2 Distributed Tracing

AWS X-Ray is instrumented across all agents, Lambda functions, and ECS tasks. Each pipeline execution carries a `trace_id` (correlated with `pipeline_id`) enabling end-to-end trace reconstruction from EventBridge trigger to SmartCOMM delivery confirmation. CloudWatch ServiceLens provides a service map view of the full agent topology.

10.3 Dashboards

- Pipeline Health Dashboard: Real-time invoice throughput, success rate, stage breakdown, DLQ depth.
- Compliance Dashboard: Validation failure categories, state-by-state compliance rates, escalation trends.
- Delivery Dashboard: SmartCOMM delivery success rate, latency distribution, fallback channel usage.
- Cost Dashboard: Bedrock LLM token spend, ECS Fargate compute cost, SQS/S3 storage costs.

11. Error Handling & Resilience

11.1 Error Taxonomy

Error Class	Severity	Handling Strategy
Transient Infrastructure	LOW	Automatic retry (3x exponential backoff, base 5s); self-healing
Data Quality	MEDIUM	Route to DLQ; LLM generates error report; analyst review queue
Compliance Failure	HIGH	Block invoice; mandatory analyst review; compliance team notification
System Outage	CRITICAL	Circuit breaker; pipeline pause; PagerDuty P1 alert; incident runbook auto-triggered
SmartCOMM API Failure	HIGH	3x retry; fallback to email delivery channel; DLQ for manual review

11.2 Idempotency Design

- Every agent operation is idempotent: pipeline_id + stage forms the idempotency key.
- S3 object keys include pipeline_id, preventing duplicate writes on retry.
- Billing System invoice records use invoice_id as a unique constraint; duplicate bind attempts return the existing record.
- SmartCOMM API calls include an idempotency-key header set to invoice_id, preventing duplicate deliveries.

11.3 Disaster Recovery

- RDS Aurora Multi-AZ: Automatic failover < 30 seconds; cross-region read replica in us-west-2 (DR region).
- S3 Cross-Region Replication: All invoice artifacts replicated to DR region with same-day RPO.
- ElastiCache: Multi-AZ with automatic failover; state loss on failover is recoverable via S3 checkpoint.
- Pipeline Replay: Failed batches can be replayed from the last successful checkpoint via S3 payload re-injection into Agent 1 queue.
- RTO: 4 hours for full pipeline recovery from regional outage. RPO: 1 hour (last checkpoint).

12. Outcomes & Measured Results

12.1 Key Performance Improvements

Metric	Before	After (Post-Launch)
Invoice generation failure rate	>95% failure	<5% (95%+ reduction)
Manual rework hours per billing cycle	800+ hrs/cycle	<40 hrs/cycle
Compliance violation rate	Unmeasured / high	0.0% detected post-validation
Invoice delivery SLA adherence	~40%	>99.5%
Time from trigger to delivery (p95)	3–7 business days	<8 hours
Policyholders served	~180K (with failures)	200,000+ (full coverage)
Regulatory audit findings	Multiple per year	Zero since deployment

12.2 Operational Impact

- Billing analysts redeployed from invoice rework to higher-value exception management and relationship-based tasks.
- Eliminated regulatory fine exposure from compliance violations estimated at \$2M+ annually.
- SmartCOMM delivery integration reduced invoice delivery lag from days to hours, improving policyholder satisfaction scores.
- Full audit trail via CloudTrail and X-Ray enabled the compliance team to pass the annual NAIC market conduct examination with zero findings.

13. Future Work & Roadmap

Phase	Feature	Description
v1.1	Real-time mode	Enable sub-hour invoice generation for on-demand billing requests via synchronous Lambda pipeline.
v1.2	Multi-LLM routing	Intelligent routing between Claude 3 Haiku (simple) and Claude 3 Opus (complex compliance) to optimize cost vs quality.
v1.3	RAG compliance	Replace static S3 rules with a Retrieval-Augmented Generation system using Bedrock Knowledge Bases for dynamic regulatory updates.
v2.0	Multi-LoB expansion	Extend pipeline to General Liability and Commercial Auto lines of business with configurable agent rule sets.
v2.1	Self-healing agents	LangGraph feedback loops enabling agents to learn from historical failures and auto-tune validation thresholds.

14. Appendix

A. Glossary

Term	Definition
LangChain	Open-source framework for building LLM-powered applications with composable chains and tool-calling agents.
LangGraph	LangChain extension for stateful, multi-actor agent workflows modeled as directed graphs.
SmartCOMM	Customer communications management platform used for regulated document generation and omnichannel delivery.
NAIC	National Association of Insurance Commissioners sets model insurance regulations adopted by US states.
WCIRB	Workers' Compensation Insurance Rating Bureau California rating and statistical organization.
DLQ	Dead Letter Queue SQS queue for messages that fail processing after maximum retry attempts.
CMK	Customer Managed Key KMS encryption key fully controlled by the customer (vs AWS-managed key).
ReAct	Reasoning + Acting LLM agent pattern that interleaves reasoning traces with tool-use actions.
SLA	Service Level Agreement contractual commitment on service performance metrics (e.g., delivery within 24 hours).

B. References

- LangChain Documentation: <https://python.langchain.com/docs/>
- LangGraph Documentation: <https://langchain-ai.github.io/langgraph/>
- AWS Well-Architected Framework: <https://aws.amazon.com/architecture/well-architected/>
- NAIC Workers Compensation Uniform Billing Standards
- AWS Bedrock Documentation: <https://docs.aws.amazon.com/bedrock/>
- SmartCOMM REST API Specification v3.2 (Internal)
- CIS AWS Foundations Benchmark v2.0

C. Document Revision History

Version	Date	Author	Changes
0.1	Jan 2026	Arch. Team	Initial draft
0.5	Feb 2026	Sec. Team	Security section added
0.9	Mar 2026	Eng. Lead	LangGraph design finalized

1.0	Apr 2026	Sr. SWE	Final review & approval
-----	----------	---------	-------------------------